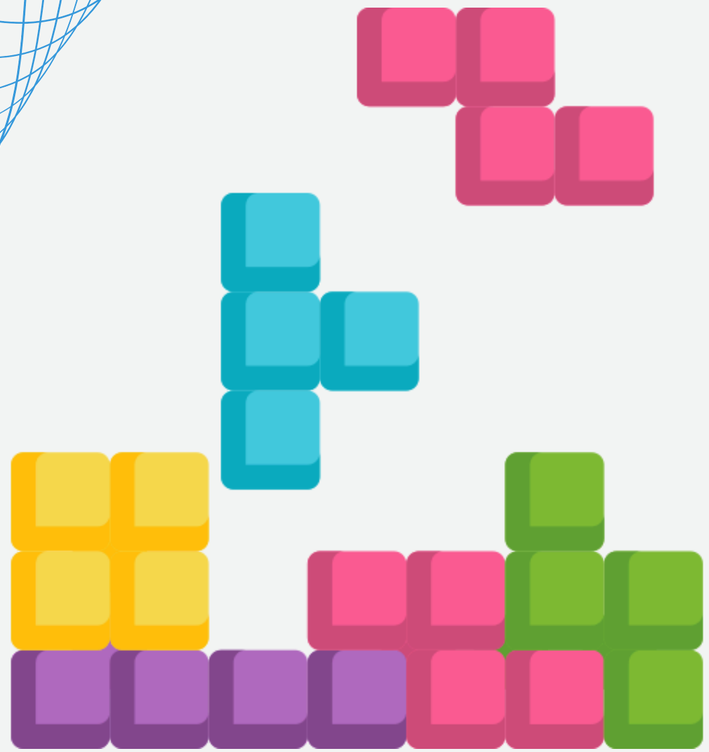
TETRIS JAVA



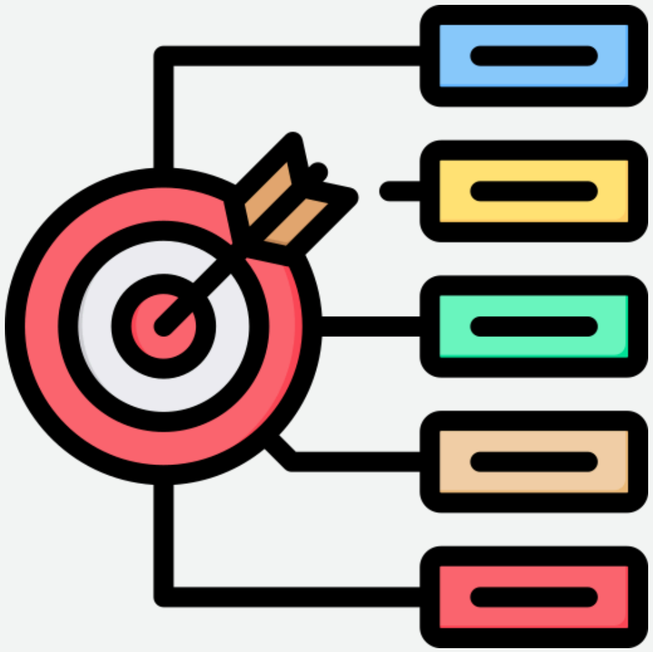
PROJECT

ที่มาและความสำคัญ



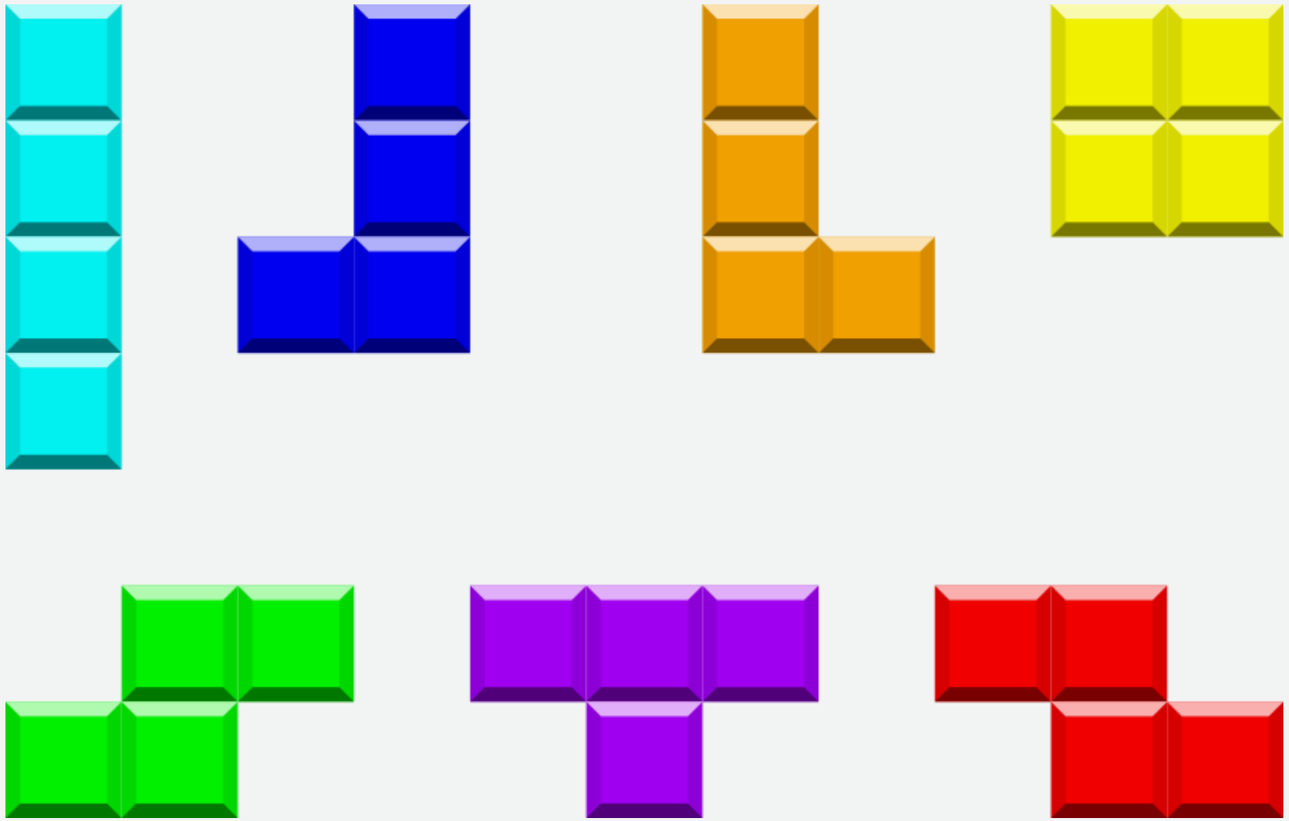
Tetris ถูกคิดค้นขึ้นโดย Alexey Pajitnov นักวิทยาศาสตร์คอมพิวเตอร์ชาวโซเวียตในปี 1984 ขณะที่เขาทำงานที่ศูนย์คอมพิวเตอร์ของ Academy of Sciences ในมอสโกว Pajitnov ได้รับแรงบันดาลใจมาจากเกมปริศนา "Pentominoes" ซึ่งเป็นเกมที่ผู้เล่นต้องจัดเรียงชิ้นส่วนรูปทรงเรขาคณิตบนกระดาน โดยใน Tetris รูปทรงเหล่านี้จะประกอบด้วย 4 บล็อกที่เรียงต่อกันในรูปทรงต่างๆ ที่เรียกว่า "tetrominoes" และผู้เล่นจะต้องวางบล็อกเหล่านี้เพื่อเติมแถวให้เต็มโดยไม่เหลือช่องว่าง

วัตถุประสงค์



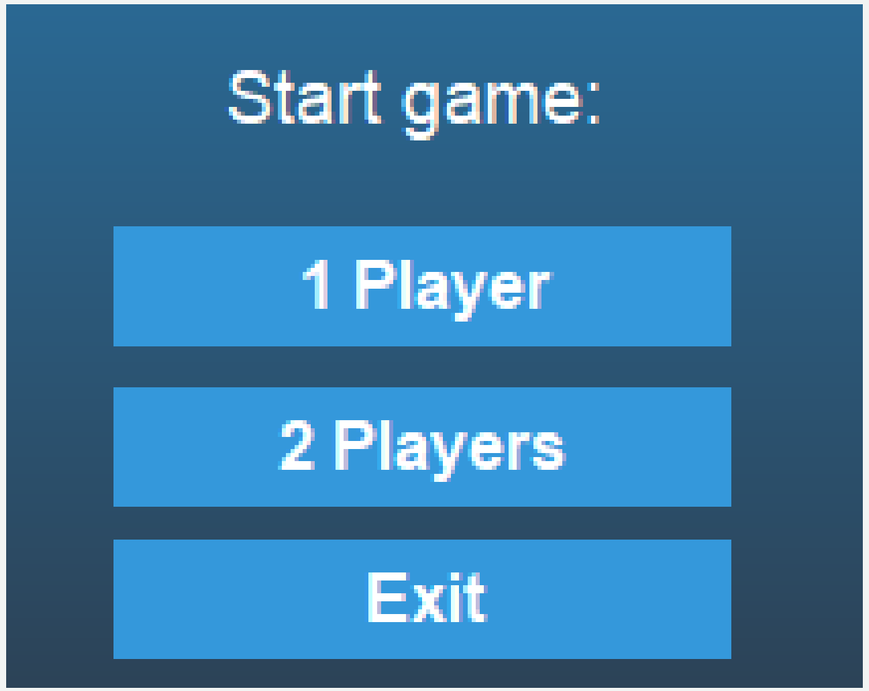
พัฒนาการคิดและเพิ่มทักษะการวิเคราะห์แก้ไขปัญหาเฉพาะหน้าและวิเคราะห์สิ่งต่างๆ ให้มีความลื่นไหลมากขึ้น ส่งเสริมพัฒนาการทางด้านต่างๆ

รูปทรงทั้ง 7 แบบ



แต่ละทรงจะมีสีที่แตกต่างกัน
เพื่อให้ผู้เล่นสามารถจำแนก
และสามารถเลือกที่จะวางตาม
รูปทรงแต่ละสีได้

การเข้าเกม



เริ่มเกมมาจะมีให้เลือก 3 เมนู คือ

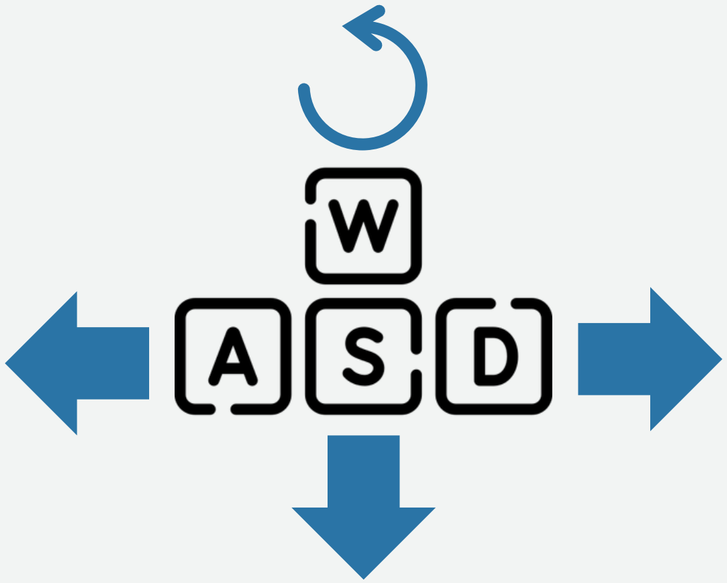
1 Player สำหรับผู้เล่นคนเดียว

2 Players สำหรับผู้เล่นหลายคน

Exit ไว้สำหรับออกหน้าต่างเกม

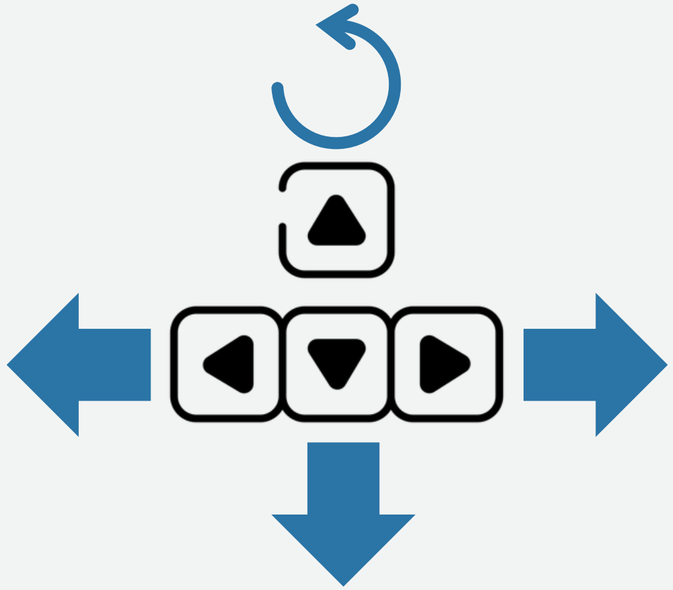
การบังคับ

สำหรับผู้เล่น1



ใช้ในการบังคับทิศทางในการเล่นเกม tetris

สำหรับผู้เล่น2



การบังคับ (ต่อ)



Quit



Reset



Pause

สำหรับผู้เล่น1



Hold



Force

สำหรับผู้เล่น2

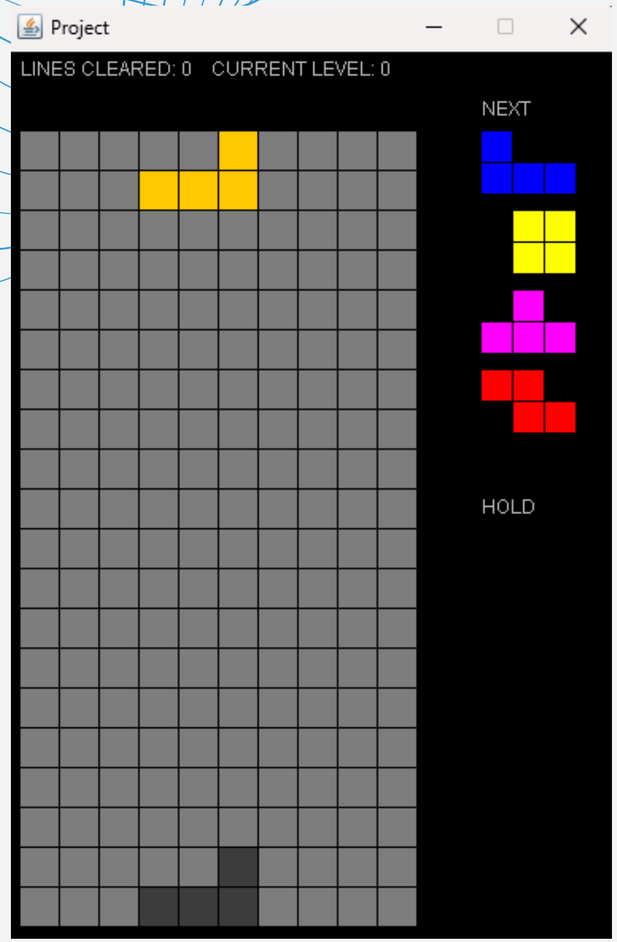


Hold



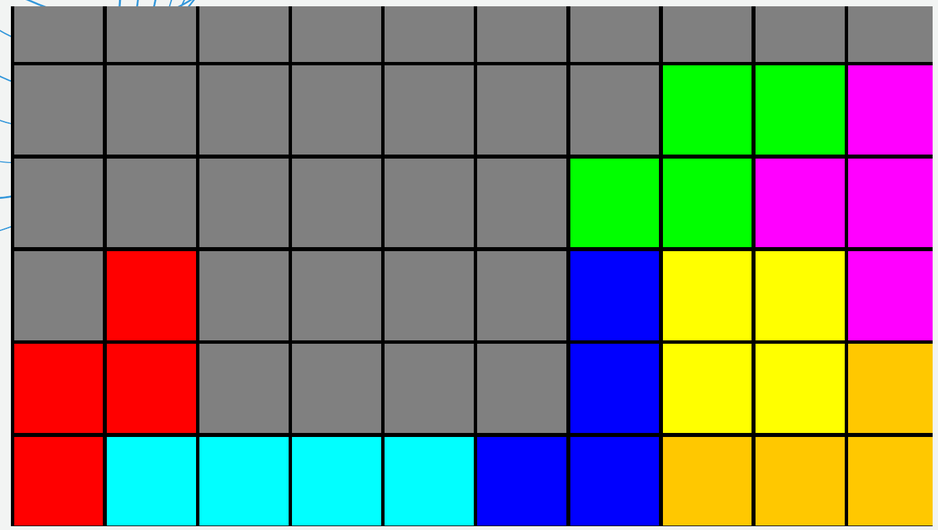
Force

วิธีการเล่น

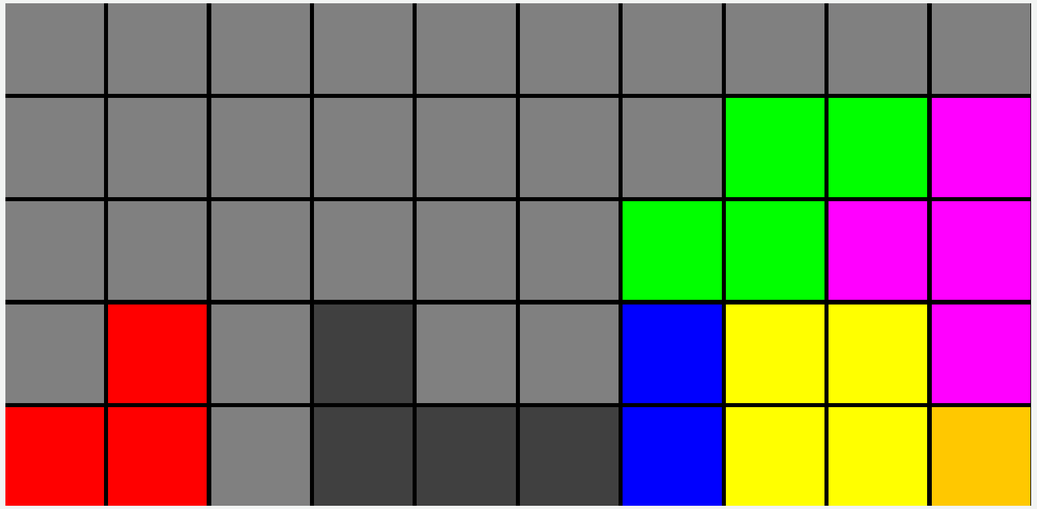


ให้เราทำการเรียงบล็อกหรือชิ้นส่วนที่เรียกว่า "tetrominoes" ซึ่งตกลงมาจากด้านบนของหน้าจอ ผู้เล่นต้องหมุนและเลื่อน tetrominoes เหล่านี้เพื่อวางลงในตำแหน่งที่เหมาะสมบนกระดานหรือกริด

วิธีการเล่น (ต่อ)

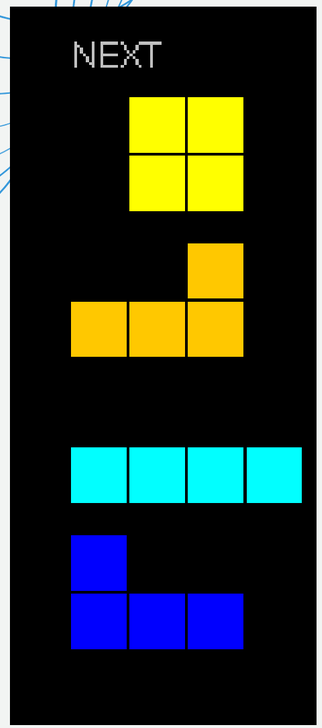


เมื่อทำการวางบล็อคเต็ม 1 แถว

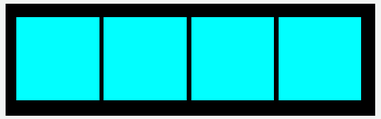


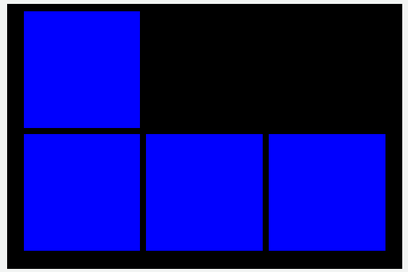
แถวตรงที่บล็อควางเต็มจะหายไป
ทันทีแล้วถูกเก็บนับเป็น score

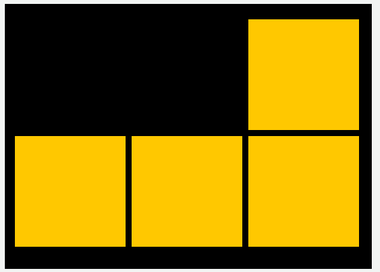
วิธีการเล่น (ต่อ)

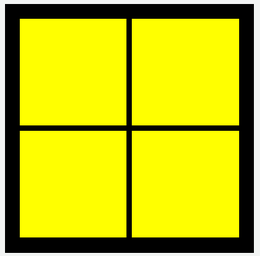


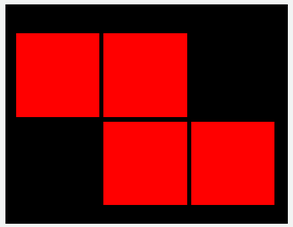
แทบด้านขวาของหน้าจอผู้เล่นจะช่วยในการบอกว่าบล็อคต่อๆ ไปรูปทรงจะเป็นแบบไหนซึ่งเราสามารถมองสีแทนได้เพราะแต่ละรูปทรงจะมีสีที่กำหนดอย่างตายตัว

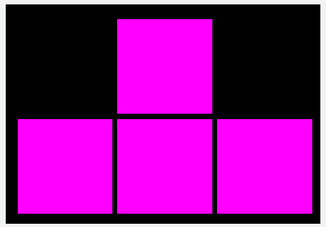


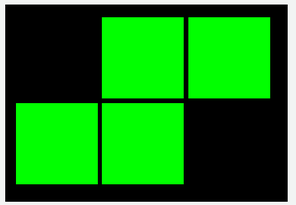




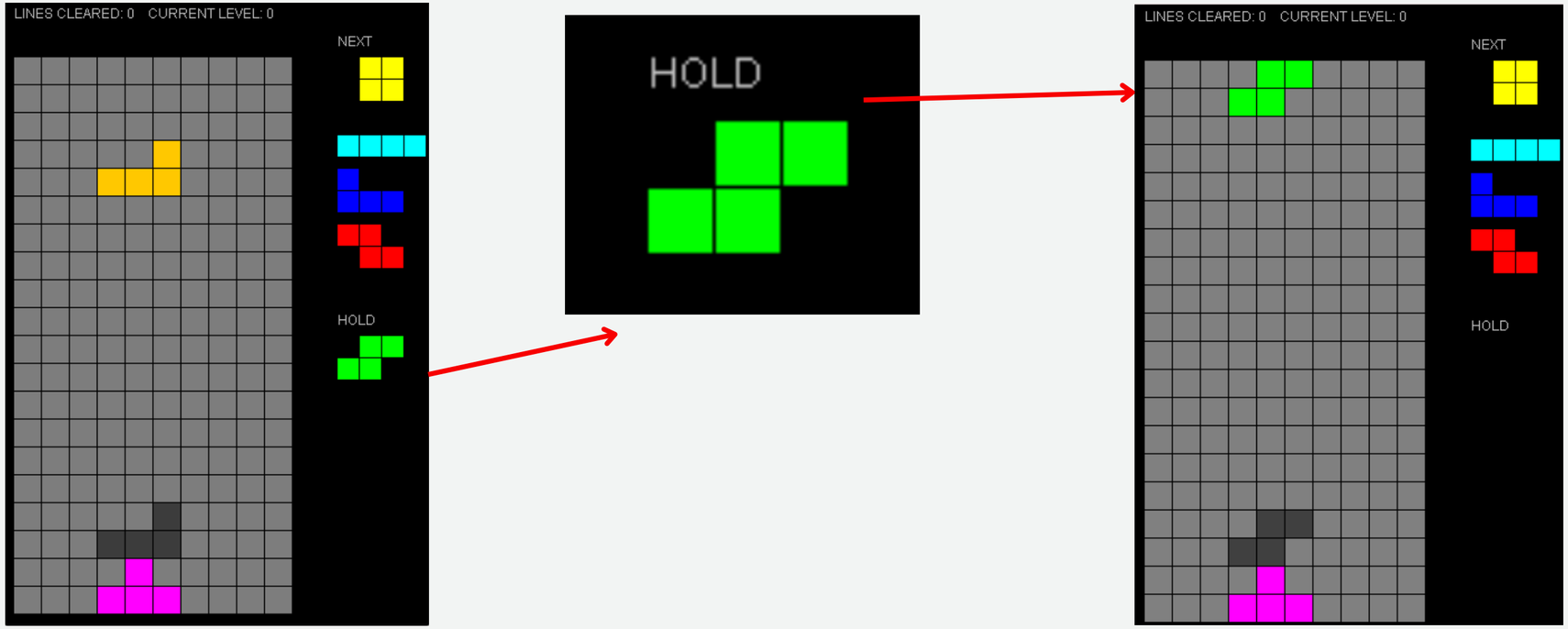








วิธีการเล่น (ต่อ)



หากเราต้องการทำการ Hold บล็อคให้กด F

หลังจากทำการกด F ตัวเกมจะนำรูปทรง Hold ลงมาแทน

ระบบเกม

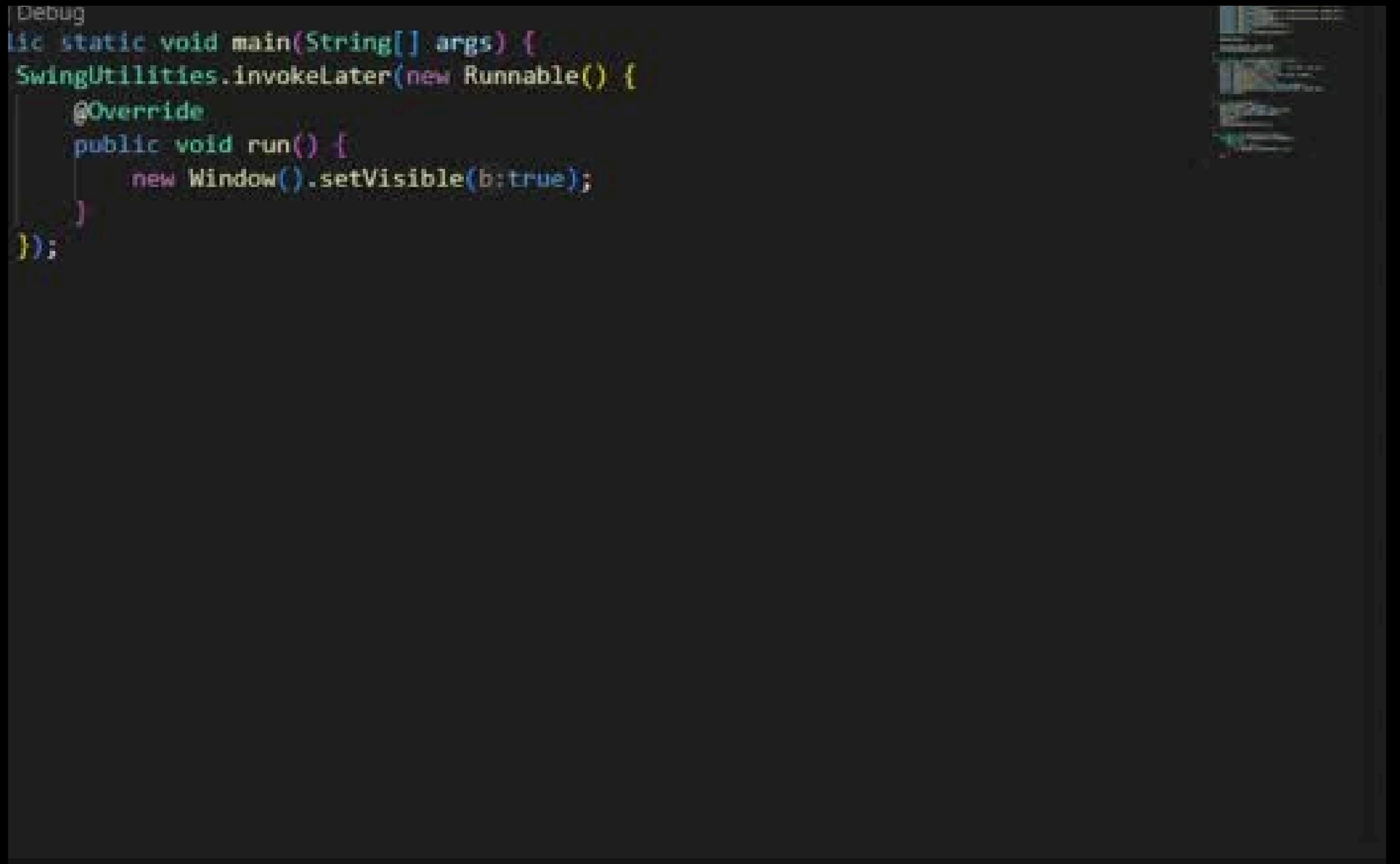




ตัวเกมจะทำการนับคะแนนหากทำการต่อครบ 1 แถวเพิ่มขึ้นทีละ 1

ตัวเกมจะทำการเพิ่ม Level ทุกๆ การต่อแถวครบ 4 แถวจะนับเพิ่มเป็น 1 Level

ทุกๆ Level จะทำให้ตัวเกมนั้นส่งบล็อคลงมาเร็วขึ้น!!


Debug
lic static void main(String[] args) {
SwingUtilities.invokeLater(new Runnable() {
@Override
public void run() {
new Window().setVisible(b:true);
}
});
TPUT
DEBUG CONSOLE
TERMINAL
PORTS
Boss\Downloads\Vs - Final T2P> & 'C:\Users\Boss\AppData\Local\Programs\Eclipse Adoptiu
sInExceptionMessages' '-cp' 'C:\Users\Boss\AppData\Roaming\Code\User\workspaceStorage\5

ประโยชน์ที่จะได้รับ



- ผู้เล่นได้รับความสนุกสนานและพัฒนาการที่ดีขึ้น
- สามารถนำเกมไปต่อยอดและคิดค้นเกมใหม่ๆ ที่มีกลิ่นอายความเป็น puzzles
- ช่วยแก้ปัญหาของคนที่มีโรคประจำตัวเช่น โรคสมาธิสั้น
- ได้กระตุ้นและพัฒนาการคิด การตัดสินใจ

การสร้างรูปทรง

```
// constants for the starting position for each piece
private final Point[][] pieces = {{mp(x:0, y:4), mp(x:0, y:5), mp(x:1, y:4), mp(x:1, y:5)},
                                  {mp(x:1, y:3), mp(x:1, y:4), mp(x:1, y:5), mp(x:1, y:6)},
                                  {mp(x:0, y:3), mp(x:1, y:3), mp(x:1, y:4), mp(x:1, y:5)},
                                  {mp(x:0, y:5), mp(x:1, y:3), mp(x:1, y:4), mp(x:1, y:5)},
                                  {mp(x:1, y:3), mp(x:1, y:4), mp(x:0, y:4), mp(x:0, y:5)},
                                  {mp(x:0, y:3), mp(x:0, y:4), mp(x:1, y:4), mp(x:1, y:5)},
                                  {mp(x:1, y:3), mp(x:1, y:4), mp(x:1, y:5), mp(x:0, y:4)}};
```

ในแต่ละแถวในอาร์เรย์ pieces แทนรูปร่างของชิ้นส่วนหนึ่งชิ้น โดยมีทั้งหมด 7 รูปแบบ ซึ่งสอดคล้องกับจำนวนชิ้นส่วนมาตรฐานในเกม

การกำหนดสี

```
private static final Color[] c = { Color.GRAY, Color.YELLOW, Color.CYAN, Color.BLUE, Color.ORANGE,
        Color.GREEN, Color.RED, Color.MAGENTA, Color.DARK_GRAY };
private static final Color ghostColor = Color.DARK_GRAY;
private static final Color UIColor = Color.LIGHT_GRAY;
```

ใช้ในการกำหนดสีว่ารูปทรงทั้ง 7 ทรงจะมีสีอะไรบ้าง

J L S T Z I O

```
// Kick cases for J L S T Z blocks
private static final int[][] movec1 = { { 0, -1, -1, 0, -1 },
        { 0, +1, +1, 0, +1 },
        { 0, +1, +1, 0, +1 },
        { 0, +1, +1, 0, +1 },
        { 0, +1, +1, 0, +1 },
        { 0, -1, -1, 0, -1 },
        { 0, -1, -1, 0, -1 },
        { 0, -1, -1, 0, -1 } };
private static final int[][] mover1 = { { 0, 0, +1, 0, -2 },
        { 0, 0, +1, 0, -2 },
        { 0, 0, -1, 0, +2 },
        { 0, 0, -1, 0, +2 },
        { 0, 0, +1, 0, -2 },
        { 0, 0, +1, 0, -2 },
        { 0, 0, -1, 0, +2 },
        { 0, 0, -1, 0, +2 } };

// Kick cases for I block
private static final int[][] movec2 = { { 0, -2, +1, -2, +1 },
        { 0, -1, +2, -1, +2 },
        { 0, -1, +2, -1, +2 },
        { 0, +2, -1, +2, -1 },
        { 0, +2, -1, +2, -1 },
        { 0, +1, -2, +1, -2 },
        { 0, +1, -2, +1, -2 },
        { 0, -2, +1, -2, +1 } };
private static final int[][] mover2 = { { 0, 0, 0, -1, +2 },
        { 0, 0, 0, +2, -1 },
        { 0, 0, 0, +2, -1 },
        { 0, 0, 0, +1, -2 },
        { 0, 0, 0, +1, -2 },
        { 0, 0, 0, -2, +1 },
        { 0, 0, 0, -2, +1 },
        { 0, 0, 0, -1, +2 } };
```

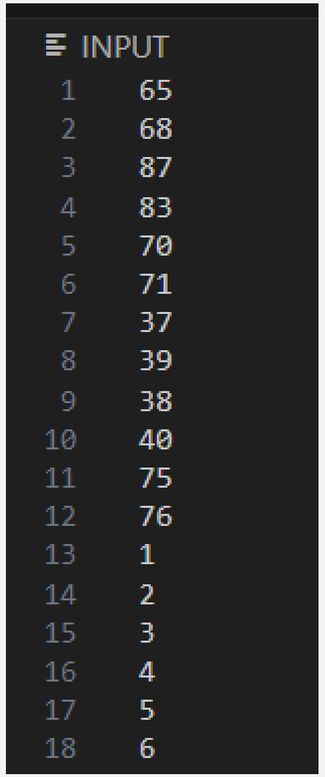
การใช้ kick cases

ใช้อาร์เรย์ movec1, mover1, movec2, mover2 เพื่อทดลองหาตำแหน่งที่เหมาะสมในกรณีที่การหมุนปกติไม่สามารถทำได้

ส่วนนี้โยงกับหลายส่วนของ class อื่นๆ

- การกำหนดค่าคงที่สำหรับ kick cases
- การจัดการกับสถานะปัจจุบันของชิ้นส่วน (curr)
- การอัปเดตตำแหน่งของชิ้นส่วนในตาราง (grid)
- การวาดชิ้นส่วนบนหน้าจอ (displayPieces())

INPUT



เป็นรหัสที่ใช้ในการเชื่อมโยงระหว่างตัวไฟล์เกมต่างๆ ใช้ในการรันโปรแกรม

การกำหนดปุ่มสำหรับแต่ละการกระทำ (เช่น เคลื่อนซ้าย เคลื่อนขวา หมุน ฯลฯ) ถูกจัดเก็บในอาร์เรย์ key โดยแต่ละผู้เล่นมีการกำหนดปุ่มของตัวเอง ซึ่งถูกอ่านจากไฟล์ "INPUT"

การสร้าง 2 หน้าจอ

```
TetrisPanel(int NUM_PLAYERS) {
    this.NUM_PLAYERS = NUM_PLAYERS;
    key = new int[NUM_PLAYERS][6];
    screens = new Tetris[NUM_PLAYERS];
    try {
        br = new BufferedReader(new FileReader(fileName:"INPUT"));
        for (int i = 0; i < NUM_PLAYERS; i++)
            for (int j = 0; j < 6; j++)
                key[i][j] = Integer.parseInt(br.readLine().trim());
    } catch (IOException ie) {
        System.exit(status:0);
    }
    addKeyListener(this);
    for (int i = 0; i < NUM_PLAYERS; i++)
        screens[i] = new Tetris(400 * i, panelR:0, this, i);
}
```

ในส่วนนี้ โปรแกรมอ่านค่าปุ่มควบคุมจากไฟล์ "INPUT" และเก็บไว้ในอาร์เรย์ key โดยแต่ละผู้เล่นมีปุ่มควบคุม 6 ปุ่ม ซึ่งรวมถึงปุ่มกี่ยวกับ K และ L ด้วย

การชน

```java
// attempts to move the active piece
// Post-condition: will return false if it cannot move and true if it can move
protected boolean movePiece(int dr, int dc) {
    if (curr == null)
        return false;
    for (Piece.Point block : curr.pos) {
        if (block.r + dr < 0 || block.r + dr >= 22)
            return false;
        if (block.c + dc < 0 || block.c + dc >= 10)
            return false;
        if (grid[block.r + dr][block.c + dc] != 0)
            return false;
    }
    for (int i = 0; i < 4; i++) {
        curr.pos[i].r += dr;
        curr.pos[i].c += dc;
    }
    curr.loc += dc;
    curr.hic += dc;
    curr.lor += dr;
    curr.hir += dr;
    return true;
}
```

เมธอด movePiece จะทำตรวจสอบว่าการเคลื่อนที่ที่ต้องการนั้นสามารถทำได้โดยไม่ชนกับขอบหรือบล็อกอื่นๆ ที่มีอยู่แล้วหรือไม่

- ตรวจสอบว่ามีชิ้นส่วนปัจจุบัน (curr) หรือไม่ ถ้าไม่มีจะคืนค่า false
- ตรวจสอบว่าจะออกนอกขอบบนหรือล่างของกริดหรือไม่
- ตรวจสอบว่าจะออกนอกขอบซ้ายหรือขวาของกริดหรือไม่
- ตรวจสอบว่าจะชนกับบล็อกที่มีอยู่แล้วในกริดหรือไม่ หากมีการชนเกิดขึ้น จะคืนค่า false

กำหนดปุ่มเกม

```java
@Override
public void keyPressed(KeyEvent e) {
    if (e.getKeyCode() == KeyEvent.VK_P) {
        boolean currentState = screens[0].isPaused;
        for (int i = 0; i < NUM_PLAYERS; i++)
            screens[i].isPaused = !currentState;
        repaint();
    } else if (e.getKeyCode() == KeyEvent.VK_Q) {
        System.exit(status:0);
    } else if (e.getKeyCode() == KeyEvent.VK_R) {
        for (int i = 0; i < NUM_PLAYERS; i++)
            screens[i].restart();
        repaint();
        return;
    }
```

ส่วนของโค้ดนี้เป็นส่วนหนึ่งของเมธอด keyPressed ซึ่งจัดการกับการกดปุ่มของผู้เล่น

กด Q
สลับสถานะการหยุดชั่วคราว (pause) ของเกมสำหรับทุกผู้เล่น
เรียก repaint() เพื่อวาดหน้าจอใหม่

กด P
สลับสถานะการหยุดชั่วคราว (pause) ของเกมสำหรับทุกผู้เล่น
เรียก repaint() เพื่อวาดหน้าจอใหม่

THE END